



INDIA.RUNS

Build what next India runs on

Team Name :

Team Name : Prince Jain

Team Leader Name :

Team Leader Name : Prince Jain

Problem Statement :

Problem Statement :

Keyword matching surfaces keyword stuffers and misses qualified people. It also can't tell if someone is available or interested. We built an offline ranker that checks real skills, catches fake profiles, and factors in platform activity.

Solution Overview

- **What is your proposed solution?**

A Python script that reads 100K candidate profiles, filters out synthetic fakes, scores the rest against the job description,

- **What differentiates your approach from traditional candidate matching systems?**

adjusts for availability signals, and outputs a ranked CSV with per-candidate reasoning. Runs in about 9 seconds on CPU. No GPU, no network, no external API calls.

What differentiates your approach from traditional candidate matching?

- We check recruiter response rates and login recency. A perfect skill match who hasn't been active in 6 months isn't useful for hiring.
- We cross-check job timelines against calendar math. If someone claims 5 years at a role that started 18 months ago, the profile gets flagged and dropped.
- Reasoning is built from actual profile fields, not generated text. Every claim in the output maps directly to a field in the candidate's JSON.

JD Understanding & Candidate Evaluation

- **What are the key requirements extracted from the JD?**
 - 5 to 9 years of hands-on ML/AI work, ideally at product companies (not consulting-only)
- **Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?**
 - Direct experience with vector databases (Pinecone, Milvus, Qdrant, FAISS), embeddings, and search evaluation metrics (NDCG, MRR, MAP)
 - Python fluency. Bonus for fine-tuning work (LoRA, PEFT). Filter out marketing, HR, sales, finance, ops roles. Pure academic researchers also deprioritized.

Which candidate signals matter most?

- Recruiter response rate: are they engaging with outreach?
- Timeline consistency: do the dates in their career history add up?
- Login recency and GitHub activity: are they active on the platform?

Ranking Methodology

- **How does your system retrieve, score, and rank candidates?**

Three sequential offline steps: (1) Honeypot filter drops synthetic profiles, (2) Feature scoring evaluates relevance, (3)

- **What models, algorithms, or heuristics are used?**

Behavioral multipliers adjust final weights.

- **How are multiple candidate signals combined into a final ranking?**

What heuristics are used?

- Technical skills match: 30% (vector DBs, embeddings, eval metrics, fine-tuning, Python)
- Title relevance: 25% (Senior/Lead AI/ML/NLP roles score highest)
- Career keywords: 25% (mentions of search, retrieval, vector, ranking in job descriptions)
- YoE fit: 20% (5-9 years = full, 4 or 10-12 = partial, outside = minimal)

How are signals combined?

Raw score gets multiplied by: response rate ($0.5 + 0.5 * RR$), login recency (1.0 down to 0.4), open-to-work flag (1.0 or 0.85), notice period (up to 1.1x for <30 days), GitHub activity. Ties break on candidate_id ascending.

Explainability & Data Validation

- **How are ranking decisions explained?**

Each candidate gets a 1-2 sentence explanation pulled from their profile: name, title, company, YoE, matched skills, signal

- **How do you prevent hallucinations or unsupported justifications?**

values like GPH score or notice period. Six sentence templates rotate to keep things varied.

- **How does your solution handle inconsistent, low-quality, or suspicious profiles?**

How do you prevent hallucinations?

There aren't any. Reasoning is assembled programmatically from parsed JSON fields. No language model is called, so there's no way for a skill or employer to appear in the reasoning that isn't in the actual data.

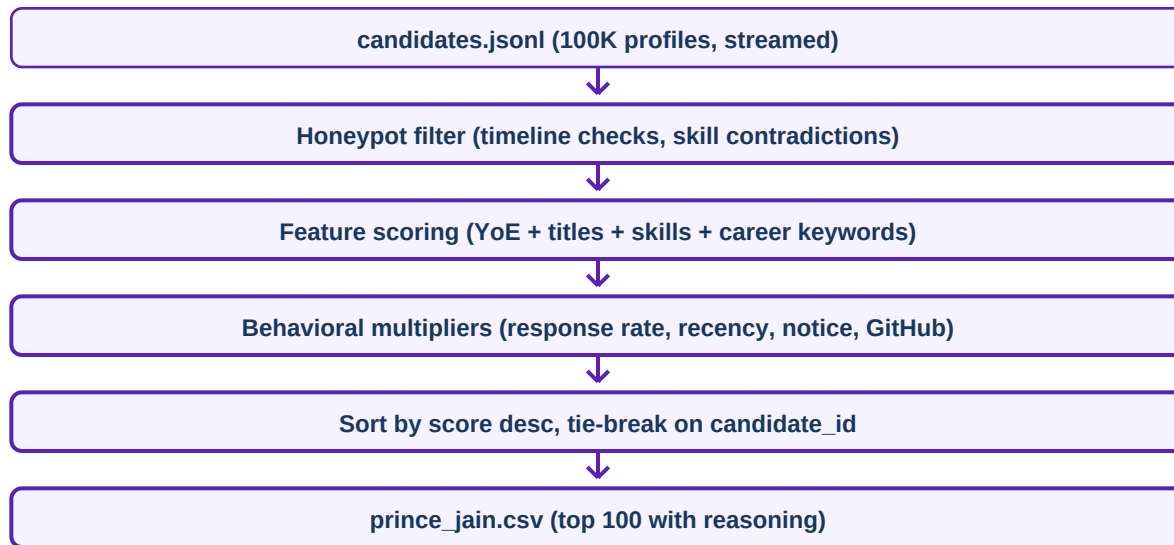
How do you handle suspicious profiles?

The honeypot filter checks three things: expert skills with zero usage duration, job durations exceeding total stated YoE, and job durations that don't fit the calendar range. Any of these trips the filter and the candidate gets excluded.

End-to-End Workflow

- Complete workflow from JD input to ranked output:
 - What is the complete workflow from JD input to ranked candidate output?
 - 1. **Load data** Stream-reads candidates.jsonl line by line. Low memory footprint.
 - 2. **Filter honeypots** Drops profiles with timeline or skill-proficiency contradictions.
 - 3. **Score relevance** Evaluates YoE, titles, skills, and career history keywords.
 - 4. **Apply modifiers** Multiplies score by response rate, login recency, notice, GitHub.
 - 5. **Sort and rank** Descending by score. Ties broken by candidate_id ascending.
 - 6. **Write output** Generates per-candidate reasoning and writes validated CSV.

System Architecture



Results & Performance

- **What results demonstrate ranking quality?**
 - Top 10 are Senior AI/ML/NLP Engineers from product companies with real vector DB and search infrastructure experience, 5-9 years in actively responding to recruiters.
- **How does your solution meet the challenge's runtime and compute constraints?**
 - Zero honeypots in the output. All synthetic profiles were caught by the filter.

Runtime and compute?

- 9 seconds end-to-end for 100K candidates on an M2 MacBook
- Under 15 MB peak memory. Well within the 16 GB constraint.
- Fully offline, no GPU, passes validate_submission.py with zero errors.

Technologies Used

- **What technologies were used and why?**
 - **What technologies, frameworks, and tools were used and why were they selected for this solution?**
 - Python stdlib (json, csv, datetime, argparse): The ranking step has zero external dependencies. Nothing to install, nothing to break, works on any Python 3.9+ environment.
 - ReportLab + pypdf: For generating this PDF using the official template. Only dependency outside stdlib, not part of the ranking pipeline.
 - Git: Version control with iterative commit history showing actual development progression.

Submission Assets

Submission assets:
Github video etc

GitHub repo:	https://github.com/princejain756/Data-AiChallengeHack2skill
Ranked output:	prince_jain.csv (100 candidates, validated)
Metadata:	submission_metadata.yaml
This deck:	approach_deck.pdf (10 slides on official template)
Entry point:	<code>python rank.py --candidates ./candidates.jsonl --out ./prince_jain.csv</code>



INDIA.RUNS

Build what next India runs on

THANK YOU